

BAB II

TEORI PENUNJANG: DASAR KONSEP GNN DAN PENDUKUNGNYA

Bab ini membahas berbagai teori yang relevan dan mendukung penyelesaian tugas akhir. Pembahasan ini bertujuan untuk memberikan pemahaman yang lebih mendalam kepada pembaca mengenai topik utama tugas akhir, serta menunjukkan bagaimana teori-teori tersebut berkontribusi dalam memahami dan menyelesaikan permasalahan yang diangkat. Melalui penjelasan yang rinci, diharapkan pembaca dapat mengenal dan memahami konsep-konsep utama yang menjadi fokus dari tugas akhir ini.

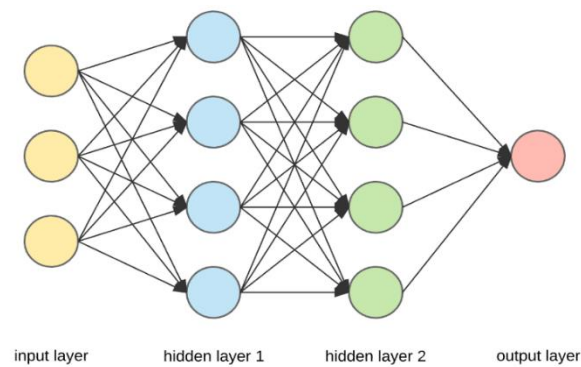
2.1 Graph

Graph, juga disebut graf, adalah struktur data yang sering digunakan dan dapat mewakili banyak hal di dunia nyata, seperti molekul, *social network*, jalan raya, koneksi internet, dan masih banyak lagi. Graf berbentuk diagram dengan node atau vertex sebagai data point dan *link* atau *edge* sebagai hubungan antara satu atau beberapa data.

2.2 Neural Network

Neural network merupakan bagian dari machine learning dan merupakan intisari dari algoritma deep learning. Neural network meniru cara kerja otak manusia di mana neuron dalam otak manusia saling mengirimkan signal satu sama lain. Sebuah neural network terdiri atas beberapa lapisan yaitu, lapisan input, lapisan tersembunyi, dan lapisan output di mana tiap lapisannya terdiri atas 1 atau lebih neuron. Tiap neuron tersebut terhubung satu sama lain. Data input dimasukkan pada neuron yang terdapat di lapisan input kemudian akan dihantarkan ke lapisan-lapisan berikutnya hingga menuju lapisan output. Sebuah neuron dapat dianggap sebagai sebuah rumus yang menerima input angka dan menyebarkan hasilnya ke neuron lainnya. Data yang diterima di lapisan input akan diproses oleh lapisan tersembunyi

dan akan dikeluarkan hasilnya pada lapisan output. Dengan mekanisme ini, neural network dapat melakukan tugas seperti pengelompokan item berdasarkan suatu label atau memprediksi label suatu item.



Gambar 2.1
Struktur Neural Network Sederhana¹

2.3 Deep Learning

Deep learning merupakan sebuah tipe dari machine learning. Yang membedakan deep learning dari machine learning adalah data yang diproses. Machine learning hanya bisa memproses data yang sudah diproses sebelumnya oleh manusia, sedangkan deep learning dapat memproses data mentahan. Data yang diproses oleh manusia bukan berarti preprocessing. Contohnya, untuk mengenali foto anjing, machine learning sudah diberitahu oleh manusia tentang fitur-fitur yang harus dicari dan dikenali. Pada deep learning, model dapat mencari tahu sendiri fitur mana yang paling menentukan seekor anjing. Deep learning bekerja dengan mencoba meniru cara kerja otak manusia. Oleh karena itu, deep learning menggunakan algoritma yang disebut dengan neural network yang akan dijelaskan lebih lanjut pada teori penunjang berikutnya. Pada tugas akhir ini, tidak akan membahas secara khusus tentang deep learning.

¹ Langkah demi Langkah Algoritma Backpropagation untuk Pemula dalam MATLAB | Pemrograman Matlab (<https://pemrogramanmatlab.com/2023/08/08/langkah-demi-langkah-algoritma-backpropagation-untuk-pemula-dalam-matlab/> diakses pada tanggal 11 Februari 2025)

2.4 Graph Neural Network

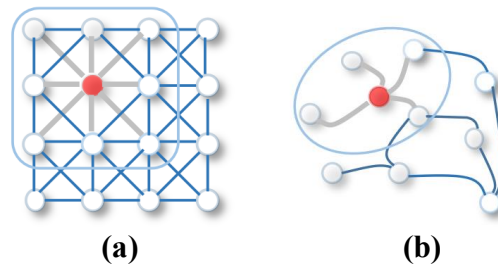
Dalam dunia pembelajaran mesin, sebagian besar model yang ada saat ini bekerja dengan data dalam bentuk tabular atau gambar. Namun, banyak data di dunia nyata yang lebih alami jika direpresentasikan dalam bentuk graf. Graph Neural Networks (GNNs) dikembangkan sebagai solusi atas keterbatasan model pembelajaran mesin konvensional yang hanya mampu menangani data dalam format grid tetap. Model seperti Convolutional Neural Networks (CNNs) menggunakan 2D convolution, yang sangat efektif dalam menangkap pola dalam data berstruktur tetap seperti gambar, namun kurang fleksibel dalam menangani data yang tidak terstruktur, seperti graf.

Berbeda dengan CNNs yang menerapkan 2D convolution dengan filter yang bergerak dalam bentuk matriks dua dimensi, GNNs menggunakan graph convolution, yang memungkinkan propagasi informasi di antara node dalam graf berdasarkan hubungan antar-node yang tidak memiliki pola tetap. Dalam graph convolution, informasi dari node tetangga dikumpulkan dan diolah untuk memperbarui representasi node yang sedang dianalisis. Hal ini memungkinkan model untuk memahami tidak hanya karakteristik node secara individual tetapi juga bagaimana koneksi dan interaksi antar-node memengaruhi representasi keseluruhan dalam jaringan.

Graph convolution memberikan fleksibilitas yang jauh lebih tinggi dibandingkan 2D convolution tradisional yang biasa digunakan dalam pemrosesan gambar. Pada model convolutional biasa, seperti CNN, struktur data yang digunakan bersifat grid tetap (misalnya gambar 2D), sehingga operasi konvolusi hanya dapat dilakukan dengan pola yang teratur dan jumlah tetangga yang konstan untuk setiap elemen. Sebaliknya, dalam graf, struktur data tidak teratur dan dinamis: setiap node bisa memiliki jumlah tetangga yang berbeda-beda, dan pola koneksi antar-node pun sangat bervariasi, tergantung pada sifat alami dari data tersebut.

Kemampuan graph convolution untuk menyesuaikan diri dengan struktur yang tidak tetap ini menjadikannya alat yang sangat powerful dalam berbagai domain aplikasi. Misalnya, dalam jaringan sosial, seseorang bisa terhubung ke lima

orang, sementara orang lain bisa terhubung ke ribuan, dan koneksi ini bisa bersifat timbal balik, bersyarat, atau berarah. Model berbasis 2D convolution tidak akan mampu menangkap kompleksitas seperti ini secara efektif.



Gambar 2.2
Perbedaan struktur 2D dan Graph Convolution² (a) 2D Convolution. (b) Graph Convolution.

2.5 Representasi Graf dalam Machine Learning

Sebelum memahami bagaimana Graph Neural Networks (GNNs) bekerja, penting untuk mengetahui bagaimana data berbentuk graf direpresentasikan dalam machine learning. Graf berbeda dengan bentuk data lain seperti tabel atau gambar karena tidak memiliki struktur yang tetap. Bentuk graf terdiri dari kumpulan entitas (node) yang saling terhubung melalui hubungan tertentu (edge).

Secara formal, sebuah graf dapat direpresentasikan sebagai:

$$G = (V, E, X) \quad \dots\dots\dots(2.1)$$

di mana:

- V adalah kumpulan node (simpul) dalam graf. Setiap node mewakili suatu entitas dalam dataset, seperti pengguna dalam jejaring sosial, molekul dalam struktur kimia, atau kota dalam jaringan transportasi.
- E adalah kumpulan edge (sisi) yang menunjukkan koneksi antara dua node. Hubungan ini bisa berupa interaksi dalam jejaring sosial, ikatan kimia antar-atom, atau jalur antara dua lokasi dalam transportasi.
- X adalah matriks fitur yang berisi informasi tambahan tentang setiap node. Contohnya, dalam jejaring sosial, fitur node dapat berupa profil pengguna.

² Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Yu, P. S. (2021). A Comprehensive Survey on Graph Neural Networks. IEEE Transactions on Neural Networks and Learning Systems hlm 2.

Dengan struktur ini, data berbentuk graf lebih kompleks dibandingkan data berbentuk tabel atau gambar karena setiap node bisa memiliki jumlah koneksi yang bervariasi dan pola hubungan antar-node tidak terstruktur secara tetap.

2.5.1 Komponen Dasar dalam Graf

Dalam dasar ilmu graf, terdapat beberapa komponen yang penting untuk dipahami agar dapat mengerti konsep graf dalam dunia *machine learning*.

1. Jenis hubungan dalam graf

Graf dapat diklasifikasikan berdasarkan arah hubungan antara node-node yang ada:

- Graf Berarah (Directed Graph)

Dalam graf berarah, hubungan antar-node memiliki arah tertentu, yang ditandai dengan panah pada setiap edge. Contohnya adalah hubungan pengikut (follower) di Twitter, di mana seorang pengguna bisa mengikuti orang lain, tetapi tidak selalu terjadi sebaliknya. Jika ada edge dari node A ke node B, ini berarti A memiliki hubungan dengan B, tetapi belum tentu B memiliki hubungan kembali dengan A.

- Graf Tidak Berarah (Undirected Graph):

Dalam graf tidak berarah, hubungan antar-node bersifat simetris, artinya jika ada edge antara node A dan B, maka hubungan itu berlaku dua arah. Contohnya adalah hubungan pertemanan di Facebook, di mana jika seseorang berteman dengan orang lain, maka orang tersebut juga otomatis menjadi temannya.

2. Sifat koneksi dalam graf

Selain berdasarkan arah hubungan, graf juga dapat dikategorikan berdasarkan kemiripan atau perbedaan karakteristik antara node-node yang terhubung:

- Graf Homofilik (Homophilic Graphs)

Dalam graf homofilik, node yang saling terhubung cenderung memiliki karakteristik yang serupa. Contohnya adalah jejaring sosial, di mana orang-orang dengan minat atau profesi yang sama lebih mungkin terhubung satu sama lain. Model GNN lebih mudah mempelajari pola dalam graf jenis ini karena

informasi dari tetangga sering kali memiliki kesamaan dengan node yang sedang dipelajari.

- **Graf Heterofilik (Heterophilic Graphs)**

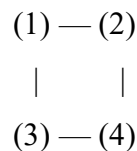
Sebaliknya, dalam graf heterofilik, node yang terhubung cenderung memiliki karakteristik yang berbeda. Contohnya adalah jaringan pelanggan dan produk dalam sistem rekomendasi, di mana pelanggan lebih mungkin terhubung dengan produk yang berbeda dari dirinya, bukan dengan pelanggan lain. Graf jenis ini lebih sulit untuk dianalisis karena informasi dari tetangga tidak selalu relevan dengan node yang sedang diproses.

2.5.2 Matriks Adjacency dan Representasi Fitur

Untuk merepresentasikan koneksi antar-node dalam graf secara matematis, graf menggunakan matriks adjacency (A). Matriks ini adalah matriks berukuran $N \times N$, di mana N adalah jumlah node dalam graf.

$$A_{ij} = \begin{cases} 1, & \text{jika ada edge antara node } i \text{ dan node } j \\ 0, & \text{jika tidak ada edge antara node } i \text{ dan node } j \end{cases}$$

Contoh terdapat graf dengan 4 node seperti di bawah ini



Matriks adjacency-nya:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Dalam contoh ini, baris dan kolom mewakili node, dan nilai 1 menunjukkan bahwa dua node memiliki hubungan langsung. Karena graf ini tidak berarah, matriks adjacency-nya simetris. Namun, jika graf berarah, maka nilai dalam matriks tidak selalu simetris karena koneksi hanya satu arah.

Selain informasi tentang koneksi antar-node, setiap node dalam graf sering kali memiliki fitur tambahan yang memberikan informasi lebih lanjut mengenai karakteristiknya. Fitur ini direpresentasikan dalam bentuk matriks fitur X , di mana

setiap baris dalam matriks menyimpan informasi fitur untuk satu node. Misalkan kita memiliki graf dengan 4 node, di mana setiap node memiliki 3 fitur, maka matriks fitur X bisa dituliskan sebagai:

$$X = \begin{bmatrix} 0.2 & 1.0 & 3.5 \\ 0.4 & 0.8 & 2.9 \\ 0.5 & 0.3 & 3.1 \\ 0.7 & 0.9 & 2.7 \end{bmatrix}$$

Setiap baris dalam matriks fitur X menyimpan representasi numerik dari karakteristik masing-masing node. Dalam konteks jaringan sosial, fitur bisa berupa usia, lokasi, atau jumlah pengikut. Dalam Graph Neural Networks (GNNs), kedua matriks ini (adjacency matrix A dan feature matrix X) digunakan secara bersamaan untuk memahami hubungan antar-node serta mengolah informasi fitur dari setiap node. Proses umum dalam GNN adalah:

1. Menggunakan matriks adjacency A untuk mengetahui bagaimana informasi dapat dipertukarkan antar-node.
2. Menggunakan matriks fitur X untuk mengetahui informasi apa yang harus dipertukarkan antar-node.
3. Menggabungkan keduanya dalam operasi propagasi untuk memperbarui representasi setiap node berdasarkan informasi dari tetangga-tetangganya.

2.6 Transformasi Linear dan Fungsi Aktivasi Non-Linear

Dalam dunia machine learning dan deep learning, dua konsep fundamental yang menjadi dasar dari hampir seluruh arsitektur jaringan saraf adalah transformasi linear dan fungsi aktivasi non-linear. Kedua komponen ini bekerja secara berurutan untuk mengubah dan memproses data agar model dapat mempelajari representasi yang kompleks dari suatu permasalahan.

Transformasi linear adalah proses matematika yang memetakan input ke output melalui operasi linier seperti perkalian matriks dan penjumlahan. Dalam neural network, biasanya direpresentasikan dengan layer yang memiliki bobot (weights) dan bias. Transformasi ini membantu model untuk mengubah representasi data mentah (seperti fitur awal dari node dalam graf) ke bentuk baru yang lebih sesuai untuk tugas selanjutnya, seperti klasifikasi atau prediksi. Namun,

transformasi linear bersifat terbatas karena hanya mampu merepresentasikan relasi linier antar variabel. Artinya, jika hanya menggunakan transformasi linear, model tidak akan bisa menyelesaikan permasalahan yang kompleks dan non-linier.

Untuk mengatasi keterbatasan tersebut, digunakanlah fungsi aktivasi non-linear. Fungsi ini diterapkan setelah transformasi linear untuk memperkenalkan non-linearitas ke dalam model. Contoh fungsi aktivasi populer adalah ReLU (Rectified Linear Unit). Fungsi ini akan menjaga nilai positif tetap sama, dan menekan nilai negatif menjadi nol. Dengan menambahkan non-linearitas, jaringan saraf menjadi mampu mempelajari pola-pola kompleks seperti hubungan antar fitur yang tidak saling proporsional atau linier. Penggabungan antara transformasi linear dan fungsi aktivasi non-linear ini merupakan blok bangunan dasar dari berbagai arsitektur deep learning, termasuk dalam Graph Neural Network (GNN).

2.7 Pembelajaran Induktif dan Transduktif

Dalam konteks pembelajaran mesin, terutama pada Graph Neural Network (GNN), penting untuk memahami perbedaan antara dua jenis pendekatan pembelajaran yang umum digunakan, yaitu pembelajaran induktif (inductive learning) dan pembelajaran transduktif (transductive learning). Perbedaan utama di antara keduanya terletak pada cakupan data saat pelatihan dan kemampuan generalisasi ke data baru.

Transductive learning adalah pendekatan di mana model dilatih dan diuji pada graf yang sama. Dalam skenario ini, semua node dan struktur graf sudah tersedia selama pelatihan, meskipun label dari beberapa node belum diketahui. Tujuan model adalah mempelajari representasi dari node yang diketahui dan menerapkannya untuk memprediksi label dari node-node yang tidak diketahui di dalam graf yang sama. Contoh nyata dari transductive learning adalah klasifikasi node di jaringan sosial, di mana kita memiliki satu graf besar (seperti graf pertemanan), dan ingin memprediksi kategori pengguna tertentu. Model tidak diharapkan untuk digunakan pada graf lain di luar data pelatihan. Karena model hanya berfokus pada satu graf yang tersedia, pendekatan ini dapat memanfaatkan informasi struktur global graf dengan sangat baik. Namun, ia memiliki keterbatasan

dalam hal generalizability, karena model tidak dirancang untuk bekerja pada graf baru yang belum pernah dilihat sebelumnya.

Sebaliknya, inductive learning adalah pendekatan di mana model dilatih untuk menggeneralisasi ke graf atau node yang belum pernah dilihat sebelumnya. Model tidak hanya belajar dari graf pelatihan, tetapi juga dikembangkan agar dapat bekerja pada data baru yang tidak disertakan selama proses pelatihan. Pendekatan ini lebih fleksibel dan banyak digunakan dalam aplikasi dunia nyata. Misalnya, jika ingin membangun sistem rekomendasi berbasis graf yang dapat diterapkan ke pengguna baru atau produk baru (yang tidak ada di graf pelatihan), maka model GNN harus menggunakan pendekatan inductive. Untuk mendukung inductive learning, model GNN biasanya tidak mengandalkan node embeddings statis, melainkan mempelajari fungsi agregasi dan transformasi fitur yang dapat diterapkan ke node manapun, di graf manapun.

2.8 Arsitektur Model GNN

Graph Neural Networks (GNN) bekerja dengan prinsip propagasi informasi, yaitu menyebarkan dan mengumpulkan informasi dari tetangga dalam graf untuk membangun representasi yang lebih kaya bagi setiap node. Berikut adalah beberapa arsitektur utama dalam GNN yang sering digunakan:

2.8.1 Graph Convolutional Network (GCN)

GCN adalah salah satu model GNN yang paling populer. Model ini menggunakan operasi konvolusi untuk menyebarkan informasi antar-node, mirip dengan bagaimana CNN bekerja pada gambar. Dalam GCN, setiap node memperbarui representasinya berdasarkan rata-rata fitur node tetangganya. Dengan cara ini, informasi dapat menyebar melalui seluruh graf selama beberapa iterasi. Model ini sering digunakan dalam klasifikasi node dan prediksi hubungan antar-node.

2.8.2 Graph Attention Network (GAT)

GAT mengembangkan pendekatan GCN dengan menambahkan bobot perhatian (attention weights) pada setiap hubungan antar-node. Perbedaannya dengan GCN adalah bahwa dalam GAT, setiap node tidak hanya mengambil informasi dari semua tetangganya secara merata, tetapi juga memberikan bobot yang berbeda tergantung pada seberapa penting hubungan tersebut. Hal ini dilakukan menggunakan mekanisme self-attention, yang memungkinkan model memprioritaskan hubungan yang lebih relevan untuk meningkatkan akurasi prediksi.

2.8.3 Graph Isomorphism Network (GIN)

GIN dirancang untuk lebih fleksibel dalam menangkap pola yang kompleks dalam data berbentuk graf. Model ini mengatasi keterbatasan GCN dalam membedakan struktur graf yang berbeda dengan menggunakan fungsi agregasi yang lebih ekspresif. Hal ini menjadikan GIN lebih unggul dalam menangkap informasi penting dari pola konektivitas dalam graf.

2.8.4 GraphSAGE

GraphSAGE adalah model GNN yang dirancang untuk menangani graf berskala besar dengan menggunakan teknik sampel tetangga untuk mengurangi kompleksitas komputasi. Alih-alih mengumpulkan informasi dari semua tetangga node, GraphSAGE memilih subset dari tetangga dan menggunakannya untuk memperbarui representasi node. Model ini sangat berguna untuk aplikasi real-world di mana graf memiliki jumlah node yang sangat besar.

2.9 Optimasi dan Regularisasi pada Pelatihan Model

Dalam pelatihan model pembelajaran mesin, terutama pada jaringan saraf seperti Graph Neural Network (GNN), terdapat dua konsep penting yang sangat berpengaruh terhadap kinerja dan keakuratan model, yaitu proses optimasi dan teknik regularisasi. Optimasi digunakan untuk menyempurnakan model dengan

cara menyesuaikan bobot-bobotnya agar kesalahan prediksi dapat diminimalkan. Sementara itu, regularisasi berperan sebagai alat pengendali kompleksitas model agar tidak terlalu menyesuaikan diri secara berlebihan terhadap data pelatihan, yang dapat menyebabkan penurunan performa pada data yang belum pernah dilihat sebelumnya (overfitting).

2.9.1 Adam Optimizer

Adam (Adaptive Moment Estimation) adalah salah satu algoritma optimasi paling banyak digunakan dalam pelatihan jaringan saraf, termasuk GNN. Pada dasarnya, ketika model membuat prediksi yang keliru, Adam membantu memperbaikinya dengan mengubah nilai bobot secara bertahap. Proses ini dilakukan berdasarkan nilai gradien, yaitu angka yang menunjukkan ke arah mana dan seberapa besar bobot harus disesuaikan untuk mengurangi kesalahan. Adam secara cerdas mengingat arah perubahan bobot dari langkah sebelumnya (disebut momentum) dan juga memperhitungkan seberapa besar rata-rata perubahan tersebut (variansi) agar pembaruan bobot menjadi lebih stabil.

Keunikan Adam terletak pada kemampuannya untuk menyesuaikan besarnya langkah pembelajaran (learning rate) secara otomatis untuk tiap bobot. Dengan cara ini, parameter yang sering berubah akan diperbarui dengan langkah kecil agar tidak melompat terlalu jauh, sementara parameter yang jarang berubah bisa diperbarui dengan langkah yang lebih besar agar lebih cepat belajar. Hasilnya, Adam membantu model untuk belajar lebih cepat dan lebih stabil, tanpa harus banyak melakukan eksperimen terhadap pengaturan parameter pelatihan.

2.9.2 Regularisasi L1 dan L2

Sementara proses optimasi bertugas menyesuaikan bobot agar hasil prediksi makin akurat, regularisasi bertugas mencegah model menjadi terlalu rumit. Jika sebuah model terlalu "menghafal" data pelatihan, ia akan bekerja sangat baik pada data tersebut, namun buruk pada data baru, ini adalah gejala overfitting.

Regularisasi membantu mencegah hal ini dengan cara memberi "hukuman" pada bobot yang terlalu besar dalam fungsi pelatihan.

Terdapat dua jenis regularisasi yang umum digunakan, yaitu L1 dan L2. Regularisasi L1 akan menambahkan total nilai absolut dari semua bobot ke dalam fungsi loss. Dengan cara ini, model terdorong untuk membuat banyak bobot menjadi nol, artinya hanya fitur yang paling penting saja yang dipertahankan. Teknik ini sering digunakan untuk seleksi fitur otomatis, karena dapat menyaring informasi yang kurang relevan. Di sisi lain, regularisasi L2 akan menambahkan total kuadrat dari bobot ke dalam fungsi loss. Teknik ini mendorong semua bobot menjadi lebih kecil tanpa benar-benar menghapusnya, sehingga model tetap mempertimbangkan semua fitur tapi dengan pengaruh yang terkontrol.

2.10 Explainable AI

Dalam model *Deep Learning*, *Artificial Intelligence* (AI) dapat dijelaskan sebagai detail atau alasan yang diberikan model agar berfungsi dengan jelas dan lebih mudah dipahami menggunakan *Explainable AI* (XAI). XAI berfungsi untuk membuat perilaku dan output keputusan sistem AI dapat dipahami oleh manusia. Tujuan *Explainable AI* bervariasi, dan beberapa di antaranya adalah sebagai berikut:

1. Kepercayaan

Keterpercayaan bisa dianggap sebagai keyakinan apakah model akan melakukan aksi yang diharapkan ketika menghadapi masalah yang diberikan.

2. Kausalitas

Explainable model memudahkan tugas untuk menemukan hubungan yang, jika terjadi, dapat diuji lebih lanjut untuk hubungan sebab akibat yang lebih kuat antara variabel yang terlibat. *Model machine learning* hanya menemukan korelasi di antara data yang dipelajarinya sehingga mungkin tidak cukup untuk mengungkap hubungan sebab-akibat.

3. Keinformatifan

Model machine learning digunakan dengan tujuan akhir untuk mendukung pengambilan keputusan. Namun, masalah yang dipecahkan oleh model tidak

sama dengan yang dihadapi oleh manusia. Oleh karena itu, banyak informasi yang dibutuhkan untuk dapat menghubungkan keputusan manusia dengan solusi yang diberikan oleh model, dan untuk menghindari kesalahan miskonsepsi. Untuk tujuan ini, *explainable model* harus memberikan informasi tentang masalah yang sedang ditangani.

4. Keyakinan

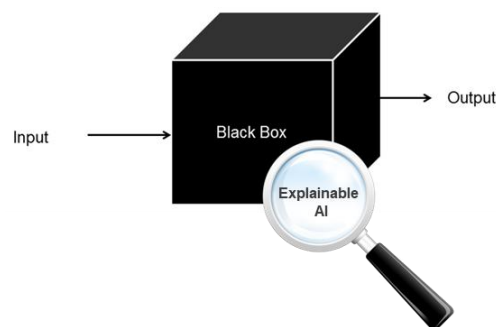
Sebagai generalisasi ketahanan dan stabilitas, keyakinan harus selalu dinilai pada model yang andal. Metode untuk mempertahankan kepercayaan di bawah kendali sangat berbeda dan tergantung pada modelnya. Oleh karena itu *explainable model* harus berisi informasi tentang kepercayaan.

5. Keadilan

Explainable model menunjukkan visualisasi yang jelas mengenai hubungan yang mempengaruhi hasil, memungkinkan untuk keadilan atau analisis etis dari model yang ada. Oleh karena itu, *explainability* harus dipertimbangkan sebagai jembatan untuk menghindari penggunaan luaran model yang tidak adil atau tidak etis.

2.11 Black Box Model

Black box model adalah istilah yang digunakan untuk menggambarkan model machine learning yang kompleks dan sulit untuk dipahami secara intuitif.



Gambar 2.4
Blackbox Model³

³ The “Black Box Effect” in Artificial Intelligence (<https://immersia.eu/en/the-black-box-effect-in-artificial-intelligence/> diakses pada tanggal 11 Februari 2025)

Dalam model ini, machine learning menghasilkan prediksi atau keputusan berdasarkan data pelatihan, tetapi internalnya sangat kompleks, sehingga sulit untuk menjelaskan bagaimana model tersebut sampai pada keputusan tersebut. Tidak jarang juga model menjadi sangat rumit sehingga manusia tidak bisa memahami bagaimana prediksi yang dihasilkan bisa tercapai. Beberapa karakteristik utama dari black box model adalah :

1. Ketidaktransparan

Black box model seringkali tidak memberikan wawasan tentang bagaimana atribut atau fitur-fitur tertentu mempengaruhi prediksi. Mereka hanyalah memberikan hasil tanpa penjelasan yang jelas.

2. Ketergantungan pada data

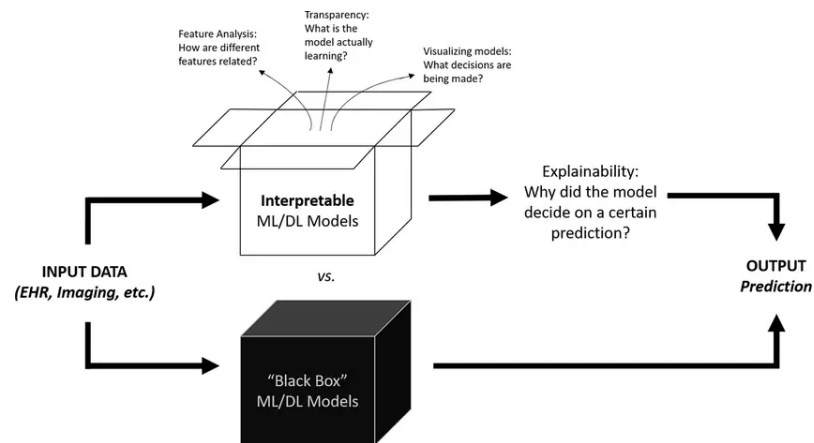
Black box model sangat bergantung pada data pelatihan yang digunakan untuk menghasilkan prediksi. Mereka tidak memiliki interpretasi yang dapat diakses manusia tentang fitur-fitur penting

3. Kompleksitas tinggi

Black box model seringkali memiliki banyak parameter dan tingkat kompleksitas yang tinggi, seperti neural network dalam deep learning. Hal ini membuatnya sulit untuk diinterpretasikan oleh manusia.

2.12 White Box Model

White box model adalah kebalikan dari black box model. Dalam white box model, seluruh proses pengambilan keputusan oleh model bersifat transparan dan dapat dijelaskan secara rinci. Artinya, baik struktur model, alur perhitungan, hingga kontribusi dari setiap fitur input terhadap hasil akhir dapat dipahami oleh manusia. Dengan demikian, white box model memungkinkan pengguna baik peneliti, analis, maupun pemangku kebijakan—untuk mengetahui secara tepat bagaimana suatu prediksi atau keputusan dihasilkan oleh model. Keunggulan utama dari pendekatan white box terletak pada aspek interpretabilitas dan kepercayaan. Dalam banyak bidang kritis seperti kesehatan, keuangan, atau hukum, memahami *mengapa* sebuah keputusan diambil sama pentingnya dengan keakuratan hasilnya.



Gambar 2.5
Whitebox Model⁴

Beberapa karakteristik utama dari white box model adalah :

1. Transparan

White box model memberikan penjelasan yang jelas tentang bagaimana setiap fitur memengaruhi prediksi. Ini memungkinkan manusia untuk memahami alur pemikiran model.

2. Ketergantungan pada penjelasan

White box model dapat diinterpretasikan oleh manusia, dan mereka sering kali dirancang untuk memberikan penjelasan yang dapat dimengerti tentang faktor-faktor yang mempengaruhi keputusan mereka.

3. Keterbatasan kompleksitas

White box model seringkali lebih sederhana dan memiliki jumlah parameter yang lebih terbatas dibandingkan dengan black box model. Ini membuatnya lebih mudah diinterpretasikan dan menjadikannya pilihan yang baik ketika interpretabilitas diperlukan.

2.13 Graf Skala-Bebas dan Preferential Attachment

Dalam banyak jaringan di dunia nyata, seperti media sosial, situs web, atau sistem rekomendasi, sering ditemukan bahwa sebagian besar titik (node) hanya

⁴ The AI Black Box: What We're Still Getting Wrong about Trusting Machine Learning Models (<https://hyperight.com/ai-black-box-what-were-still-getting-wrong-about-trusting-machine-learning-models/> diakses pada tanggal 11 Februari 2025)

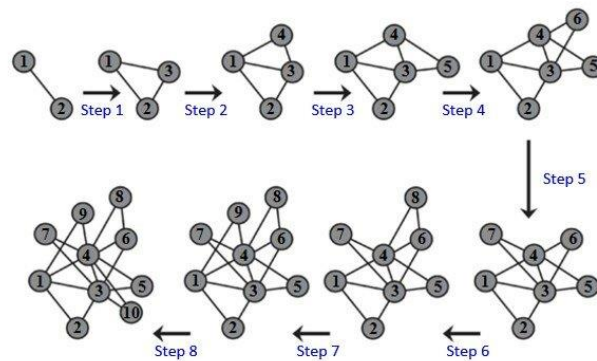
memiliki sedikit koneksi, sedangkan beberapa titik lainnya memiliki sangat banyak koneksi. Jaringan dengan pola seperti ini disebut graf skala-bebas (scale-free graph). Disebut "skala-bebas" karena jumlah koneksi (derajat) pada node-node dalam graf ini mengikuti pola yang disebut distribusi power-law. Artinya, sebagian besar node hanya terhubung ke satu atau dua node lain, sementara sebagian kecil node bertindak sebagai “pusat” yang memiliki banyak koneksi. Fenomena ini berbeda dengan graf acak, di mana setiap node memiliki peluang yang hampir sama untuk memiliki banyak atau sedikit koneksi, sehingga distribusinya lebih merata seperti lonceng (normal distribution).

Salah satu alasan mengapa graf skala-bebas bisa terbentuk adalah karena adanya mekanisme yang disebut preferential attachment. Mekanisme ini menjelaskan bahwa node baru yang masuk ke dalam jaringan cenderung lebih suka terhubung ke node yang sudah populer atau memiliki banyak koneksi sebelumnya. Contoh nyata bisa dilihat di media sosial, akun dengan banyak pengikut biasanya akan lebih mudah mendapatkan pengikut baru dibanding akun yang tidak dikenal. Mekanisme inilah yang menjadi dasar dari model pembentukan graf yang disebut Barabási–Albert Model.

Dalam model Barabási–Albert, proses pembentukan jaringan dimulai dengan beberapa node awal. Setiap kali node baru ditambahkan, ia akan memilih untuk terhubung ke node-node yang sudah ada dengan kemungkinan yang proporsional terhadap jumlah koneksi yang dimiliki node tersebut. Jika satu node sudah memiliki banyak koneksi, maka kemungkinan ia akan dipilih oleh node baru menjadi lebih besar. Seiring waktu, hal ini membuat beberapa node memiliki koneksi yang sangat banyak, dan menciptakan struktur jaringan yang tidak seimbang.

Graf skala-bebas sangat penting dalam studi pembelajaran mesin berbasis graf, khususnya pada model seperti Graph Neural Network (GNN). Struktur seperti ini sering dijumpai dalam data nyata, sehingga model GNN harus mampu bekerja dengan baik meskipun ada ketimpangan dalam jumlah koneksi antar node. Node yang menjadi pusat (hub) bisa menyebarkan informasi ke banyak node lainnya, dan hal ini dapat memengaruhi bagaimana informasi diproses dan dipelajari oleh model.

Oleh karena itu, banyak eksperimen GNN menggunakan graf skala-bebas, baik dari data nyata maupun dari graf buatan (sintetis) yang dibentuk menggunakan model Barabási–Albert.



Gambar 2.6
Proses Terbentuknya Graf Skala-Bebas Melalui Mekanisme Preferential Attachment⁵

Secara keseluruhan, memahami graf skala-bebas dan mekanisme preferential attachment sangat penting untuk membangun model pembelajaran yang lebih akurat dan relevan dengan dunia nyata. Preferential attachment membantu kita menjelaskan bagaimana jaringan bisa tumbuh secara alami, dan graf skala-bebas membantu kita memodelkan kondisi yang lebih realistis dalam berbagai aplikasi berbasis graf.

⁵ The emergence of a scale free network. ResearchGate (https://www.researchgate.net/figure/The-emergence-of-a-scale-free-network-can-be-observed-when-newly-added-edges-show-a-high_fig2_310261624 diakses pada tanggal 11 Februari 2025)